

Exercise 7.1: Apply Newton's method to find all three solutions of

$$f(x) = x^3 - 5x^2 - 12x + 19 = 0$$

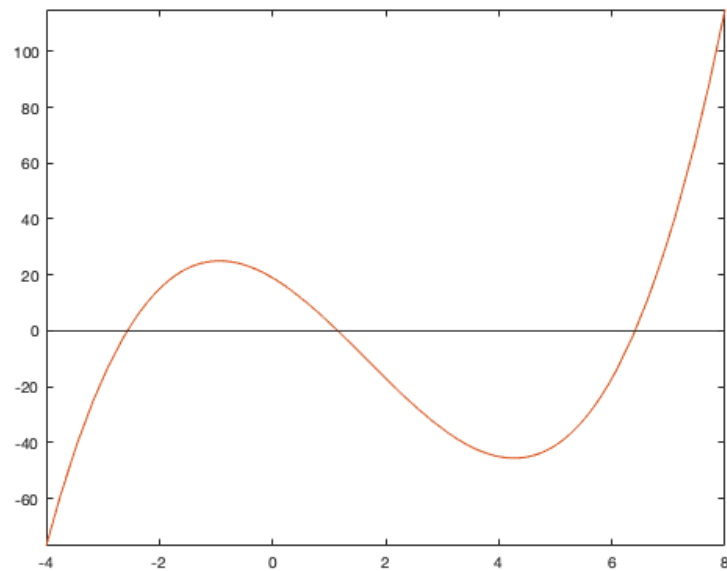
You will have to use several different initial guesses.

Solution. First we need to compute the derivative, so we get

$$f'(x) = 3x^2 - 10x - 12.$$

Plotting the function to get a good idea for initial guesses,

Figure 1: Plot of $f(x)$.



Plugging in $x_0 = -2$ we get the following iterates. Note that we achieve 16 digit of accuracy on $x_5 = -2.565244464949796$.

Console:

```
>> f = @(x) x.^3 - 5*x.^2 - 12*x + 19
>> df = @(x) 3*x.^2 - 10*x - 12

>> x = -2
x =
    -2

>> x = x - f(x)/df(x)
x =
   -2.7500000000000000

>> x = x - f(x)/df(x)
```

```

x =
-2.576923076923077

>> x = x - f(x)/df(x)
x =
-2.565295955416702

>> x = x - f(x)/df(x)
x =
-2.565244465957728

>> x = x - f(x)/df(x)
x =
-2.565244464949796

-----
>> x = x - f(x)/df(x)
x =
-2.565244464949796

```

Plugging in $x_0 = 1$ we get the following iterates. Note that we achieve 16 digits of accuracy on $x_4 = 1.155546012813802$.

Console:

```

>> f = @(x) x.^3 - 5*x.^2 - 12*x + 19
>> df = @(x) 3*x.^2 - 10*x - 12

>> x = 1
x =
1

>> x = x - f(x)/df(x)
x =
1.157894736842105

>> x = x - f(x)/df(x)
x =
1.155546444013717

>> x = x - f(x)/df(x)
x =
1.155546012813817

>> x = x - f(x)/df(x)
x =
1.155546012813802

-----
>> x = x - f(x)/df(x)
x =
1.155546012813802

```

Plugging in $x_0 = 6$ we get the following iterates. Note that we achieve 15 digits of accuracy on $x_6 = 6.409698452135994$.

Console:

```
>> f = @(x) x.^3 - 5*x.^2 - 12*x + 19
>> df = @(x) 3*x.^2 - 10*x - 12
```

```
>> x = 6
x =
    6
```

```
>> x = x - f(x)/df(x)
x =
    6.472222222222222
```

```
>> x = x - f(x)/df(x)
x =
    6.410844871828556
```

```
>> x = x - f(x)/df(x)
x =
    6.409698848505079
```

```
>> x = x - f(x)/df(x)
x =
    6.409698452136041
```

```
>> x = x - f(x)/df(x)
x =
    6.409698452135993
```

```
>> x = x - f(x)/df(x)
x =
    6.409698452135994
```

```
>> x = x - f(x)/df(x)
x =
    6.409698452135995
```

```
-----
Alternates Between
```

```
x =
    6.409698452135994
x =
    6.409698452135995
```

□

Exercise 7.10: Apply Newton's method to the system of nonlinear equations,

$$f_1(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0$$

$$f_2(x_1, x_2) = 5x_1^2 - x_2 - 2 = 0$$

There are four solutions to this system of equations. Can you find all four of them by using different initial guesses?

Solution. Applying Newton's Methods for systems of nonlinear equations we get the following iteration step,

$$x_{n+1} = x_n - (\nabla f(x_n)^T)^{-1} f(x_n).$$

Where $(\nabla f(x_n)^T)^{-1}$ is the inverse of the Jacobian. First let's define $\nabla f(x_n)^T$ for our system,

$$\nabla f(x_n)^T = \begin{pmatrix} 2x_1 & 2x_2 \\ 10x_1 & -1 \end{pmatrix}.$$

Using Matlab we can evaluate the jacobian and the function at each iterate and perform Newton's Method. The following is a function which takes both functions, an initial value, and an x-tolerance and performs Newton's Method.

Code:

```
function [Hist, Val] = NewtonMethodMulti(f, g, xinit, xtol)
% This function takes a two function of the form
% f = x + y + c where c is a constant, an initial value
% xinit in the form of a column vector, and an xtolerance
% for computing Newton's method for a system of linear equations
% with two variables and two equations.

%Symbolically computing the jacobian
syms x y
Jacobian = jacobian([f, g], [x, y]);

%Evaluating the functions and jacobian
fx1 = double(subs(f, [x y], xinit'));
gx1 = double(subs(g, [x y], xinit'));
Jacobianx1 = double(subs(Jacobian, [x y], xinit'));

%First Newton step
x1 = xinit - inv(Jacobianx1)*[fx1; gx1]; % Column vector

Hist = [xinit x1]; % Hist each col is an iteration
Count = 2;

while(norm(Hist(:, Count) - Hist(:, (Count - 1))) >= xtol)
    xn = Hist(:, Count); % Assign current iterate

    % Compute function values and Jacobian
    fxn = double(subs(f, [x y], xn'));
    gxn = double(subs(g, [x y], xn'));
    Jacobianxn = double(subs(Jacobian, [x y], xn'));
```

```
% Compute next iterate
xn = xn - inv(Jacobianxn)*[fxn;gxn];

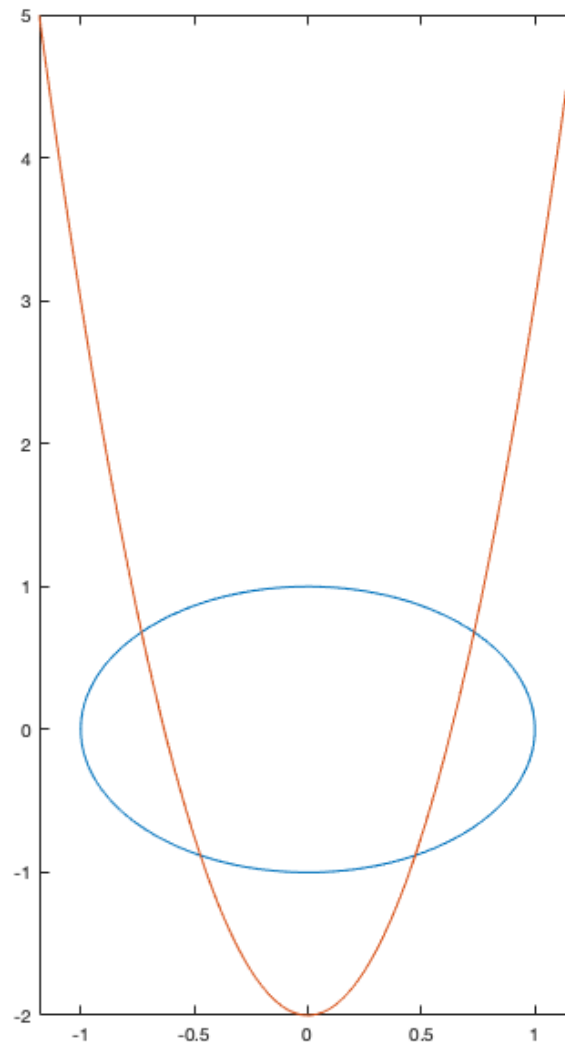
% Append Hist iterate Count
Hist = [Hist xn];
Count = Count + 1;
end

% Return final iterate
Val = Hist(:, Count);

% Sanity Check
fxfinal = double(subs(f, [x y], Val'))
gxfinal = double(subs(g, [x y], Val'))

end
```

Plotting both functions we get the following,

Figure 2: Plot of f_1 and f_2 .

Where these two functions intersect are points where both functions are satisfied, so let's pick our initial values around there. Doing so we get $x_0 = (-.5, -.9)$, $x_0 = (-.7, .7)$, $x_0 = (.5, -.9)$, $x_0 = (.7, .7)$. Using these values as the input to our Newton routine we get,

Console:

```
>> [Hist, Val] = NewtonMethodMulti(f, g, [-.5; -.9], eps)
```

Hist =

```
-0.5000000000000000    -0.9000000000000000
-0.4737500000000000    -0.8812500000000000
```

```

-0.473070250659631  -0.8810250000000000
-0.473069769148377  -0.881024967590666
-0.473069769148132  -0.881024967590665
-0.473069769148132  -0.881024967590665

```

Val =

```

-0.473069769148132
-0.881024967590665

```

```
>> [Hist , Val] = NewtonMethodMulti(f , g , [ -.7;.7] , eps)
```

Hist =

```

-0.7000000000000000  0.7000000000000000
-0.733035714285714  0.6812500000000000
-0.732260609883417  0.6810250000000000
-0.732260195230019  0.681024967590666
-0.732260195229901  0.681024967590665
-0.732260195229901  0.681024967590665

```

Val =

```

-0.732260195229901
0.681024967590665

```

```
>> [Hist , Val] = NewtonMethodMulti(f , g , [.5;-.9] , eps);
```

Hist =

```

0.5000000000000000  -0.9000000000000000
0.4737500000000000  -0.8812500000000000
0.473070250659631  -0.8810250000000000
0.473069769148377  -0.881024967590666
0.473069769148132  -0.881024967590665
0.473069769148132  -0.881024967590665

```

Val =

```

0.473069769148132
-0.881024967590665

```

```
>> [Hist , Val] = NewtonMethodMulti(f , g , [.7;.7] , eps);
```

Hist =

```

0.7000000000000000  0.7000000000000000
0.733035714285714  0.6812500000000000
0.732260609883417  0.6810250000000000
0.732260195230019  0.681024967590666
0.732260195229901  0.681024967590665

```

0.732260195229901 0.681024967590665

Val =

0.732260195229901
0.681024967590665

□

Exercise 2.4: Prove Corollary 6.7. If x is a feasible solution to the primal, y is a feasible solution to the dual, and $c^T x = b^T y$, then x and y are optimal for their respective problems.

Proof. Suppose x is a feasible solution to the prime LP problem, y is a feasible solution to the dual LP problem, and $c^T x = b^T y$. By Weak Duality we know that for all feasible solutions h in the primal and g in the dual, $c^T h \leq b^T g$. Therefore for all feasible solutions h in the primal we know that $c^T h \leq b^T y = c^T x$. So we get that for all feasible solutions h in the primal $c^T h \leq c^T x$ and thus x must be the optimal solution in the primal.

Similarly, for all feasible solutions g in the dual we get that $b^T g \geq c^T x = b^T y$. So we get $b^T g \geq b^T y$ for all feasible solutions g in the dual and thus y must be optimal solution in the dual. □

Exercise 2.8: Using duality theory find the solutions to the following linear programs,

$$\begin{aligned} \text{Minimize } z &= x_1 + 2x_2 + \cdots + nx_n \\ \text{Subject to } x_1 &\geq 1 \\ x_1 + x_2 &\geq 2 \\ &\vdots \\ x_1 + x_2 + \cdots + x_n &\geq n \\ x_1, x_2, \dots, x_n &\geq 0 \end{aligned}$$

Proof. First let's convert the given LP problem into standard form. Doing so we get the

following,

$$\begin{aligned}
 &\text{Minimize } z = x_1 + 2x_2 + \cdots + nx_n + \sum_{i=1}^n e_i \\
 &\text{Subject to } x_1 + s_1 = 1 \\
 &\quad x_1 + x_2 + s_2 = 2 \\
 &\quad \vdots \\
 &\quad x_1 + x_2 + \cdots + x_n + s_n = n \\
 &\quad x_1, x_2, \dots, x_n, s_1, s_2, \dots, s_n \geq 0
 \end{aligned}$$

Note that the matrix A that we have formed looks like $A = [L_1 I_n]$ where L_1 is a lower triangular matrix with all ones. Generating the dual problem we get the following, and note that A^T now looks like an upper triangular matrix stacked on top of an identity matrix,

$$\begin{aligned}
 &\text{Maximize } w = y_1 + 2y_2 + \cdots + ny_n + \sum_{i=1}^n e_i \\
 &\text{Subject to } y_1 + y_2 + \cdots + y_n \leq 1 \\
 &\quad y_2 + y_3 + \cdots + y_n \leq 2 \\
 &\quad \vdots \\
 &\quad x_n \leq n \\
 &\quad s_1, s_2, \dots, s_n \leq 1
 \end{aligned}$$

Now consider the feasible solution in the primal, $x = (n, 0, \dots, 0)$ with excess variables set to zero, which gives a function value of $z = c^T x = n$. Also consider the feasible solution in the dual, $y = (0, 0, \dots, 1, 0, \dots, 0)$ where the one corresponds to x_n gives us $w = b^T y = n$. Since we have two feasible solutions in their respective problems, that also have the property that $c^T x = b^T y$ they must be optimal in their respective problems. Therefore we get that $x = (n, 0, \dots, 0)$ is the optimal solution to the original problem (excess zeroes removed). $\square$

Exercise 2.11: Prove that if the problem L_1

$$\begin{aligned}
 &\text{Minimize } z = c^T x \\
 &\text{Subject to } Ax = b \\
 &\quad x \geq 0
 \end{aligned}$$

has a finite optimal solution, then the new problem L_2

$$\begin{aligned}
 &\text{Minimize } z = c^T x \\
 &\text{Subject to } Ax = \hat{b} \\
 &\quad x \geq 0
 \end{aligned}$$

cannot be unbounded for any choice of the vector $\hat{b}$.

Proof. Suppose for the sake of contradiction that there exists a $\hat{b}$ such that L_2 is unbounded. Choose the said $\hat{b}$ and construct the dual of L_2 , namely D_2 . Note that the constraints of D_2 are denoted by $A^T y \leq c$. By Corollary 6.6, since L_2 is unbounded, we know that D_2 is infeasible and by definition there exists no solution for $A^T y \leq c$. Now consider the dual of L_1 , namely D_1 and note that it has the same constraints as D_2 i.e $A^T y \leq c$. By Strong Duality since L_1 has an optimal solution, then D_1 also has an optimal solution, y^* . Note that since y^* is optimal it must satisfy $A^T y^* \leq c$. Therefore y^* is also a feasible solution to D_2 . Thus a contradiction. $\square$

Exercise Problem P10: Suppose we have an LP problem in standard form, and we do not have a basic feasible solution. Write code which implements the phase-1 strategy for generating an initial basic feasible solution.

Solution. The following code sets up the new phase one LP problem. We must add the m artificial variables to the matrix A in the form of an $m \times m$ identity matrix so our new A looks like $[A I_m]$. The goal is to minimize the artificial variables so we end up with $c = (0, \dots, 0, 1, \dots, 1)$ a vector with n zeros followed by m ones. An initial basic feasible solution can then easily be constructed by $x_0 = (0, \dots, 0, b_1, b_2, \dots, b_m)$. We report that the problem is unbounded if the value of our objective function for the phase one problem is greater than zero.

Code:

```
function x = phaseone(A,b)
    % Pull Sizes
    [m, n]= size(A);
    % Generate new phase one matrix
    A1 = [A eye(m)];
    % Generate phase one objective function
    c1 = [zeros(n, 1);ones(m, 1)];
    % Generate phase one feasible solution
    b1 = b(:);
    x1 = [zeros(n, 1);b];

    % Running simplex on phase one problem
    [x, z1] = sfsimplex(c1,A,b1,x1 ,true);

    % Reporting if problem is infeasible
    % no solution to Ax = b in the original problem
    if z1 > 0
        error('Infeasible ')
    else
    end
```

$\square$

Exercise Problem P11: Use the provided scripts to run the simplex method on a Klee-Minty cube of size n . On your machine how big of an n can you run in under 20 seconds? For this maximum n , how many iterations does it take to find the optimal solution? What are the number of constraints and the number of variables if you were to put the problem in standard form?

Solution. On my machine size $n = 13$ took 19.3544 seconds to run, and $n = 14$ took about 60 seconds. For $n = 13$ it took 8192 iterations of the simplex method to find the optimal solution. The Klee-Minty cube as defined in the comments of Klee-Minty.m is an LP problem with n variables, $2n$ inequality constraints. When we transform it into standard form n slack variables are added, so we end up with $2n$ variables, n equality constraints, and $2n$ nonnegativity constraints. $\square$